

# Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Notation</b>                             | <b>1</b> |
| <b>2</b> | <b>Definition</b>                           | <b>1</b> |
| <b>3</b> | <b>Limitations/ Assumptions</b>             | <b>1</b> |
| <b>4</b> | <b>Tail Recursion Elimination Algorithm</b> | <b>1</b> |
| 4.1      | short . . . . .                             | 1        |
| 4.1.1    | funcOp . . . . .                            | 2        |
| 4.1.2    | callOp . . . . .                            | 2        |

## 1 Notation

- [...] sequence of zero or more elements.

## 2 Definition

**Definition 2.1** (Tail Recursive Call). *A call from procedure  $f( )$  to procedure  $g( )$  is a tail call if the only thing  $f( )$  does, after  $g( )$  returns to it, is itself return. The call is tail-recursive if  $f( )$  and  $g( )$  are the same procedure.[1]*  
*Adaption for our purpose:*  
*Still tail recursive call, while nested operations with regions or blocks return. i.e. Operations with OpTrait ReturnLike and IsTerminator.*

## 3 Limitations/ Assumptions

1. Controllflow only sequential(line after line) or through the scf-Dialect. So no cf/ llvm jumps / arith select(? might be possible to allow arith select. as we still have to have a topMostIf)
2. Recursive call nested within a scf::ifOp. By (1.) If recursive call is not nested within scf::ifOp Recursion would never terminate. WRONG: What about while and for?
3. func.func is tail recursive and valid MLIR
4. Set of allowed dialects for Operations is arith, scf, sigi, (closure, func)

## 4 Tail Recursion Elimination Algorithm

### 4.1 Inputs

Input to the Algorithm:

#### 4.1.1 funcOp

Operation of type `func::funcOp` and the form:

```
func.func @sym_name (%arg_i | i in {1...n}; type
    in {i32, i1, sigi.stack})
    -> [resultstypes; have same type as args]
1. [Operation | in allowed Dialects] = PROTO_HEAD
2. scf::ifOp %cond -> ? / = resultstypes guranteed? {
    THENBODY
  } else {
    ELSEBODY
  }
3. func.return %result: resultttypes
```

Where either

*THENBODY* = *PROTO\_WHILEBODY* and *THENBODY* = *PROTO\_TERMINATION*  
or *THENBODY* = *PROTO\_TERMINATION* and *THENBODY* = *PROTO\_WHILEBODY*.

With *PROTO\_WHILEBODY* =

```
1. [Operation | in allowed Dialects]
2. func.call @sym_name(operands;) -> resultttypes
3. scf::yield %yieldRes : ? / = resultstypes
   guranteed?
4. [Operation | in allowed Dialects and with
   OpTrait ReturnLike or IsTerminator]
```

and *PROTO\_TERMINATION* =

```
1. [Operation | in allowed Dialects]
2. scf::yield %yieldRes : ? / = resultstypes
   guranteed?
```

#### 4.1.2 callOp

Operation of type `func::CallOp` and the form

```
func.call @sym_name(funcOp.operands;) ->
    funcOp.resultttypes
```

For which holds true that

1. Operation referenced by `sym_name` in `SymbolTable` is `funcOp`. I.e. `callOp` calls `funcOp`.
2. `callOp` can be found when walking over `funcOp`.
3. All owners of uses of `callOp`'s results have `OpTrait ReturnLike` or `IsTerminator`

4. (3) Holds also holds true for uses of owners of uses of callOp's result. I.e. any operation dependent on callOp's result has OpTrait ReturnLike or IsTerminator
5. All Operations in callOps block after callOp are Pure. (If dependent on callOp's results (3) and (4) have to hold. If not dependent on callOp's result, can get moved before callOp)
6. All operations dependent on results of ops in 5 are also Pure

(1) and (2) set the requirement that callOp is a recursive call. 3 - 5 set the requirement that callOp is located at the tail of funcOp.

## References

- [1] S. S. Muchnick, *Advanced compiler design and implementation*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1998.